

Universidad de los Andes

Departamento de Ingeniería Sistemas y Computación

ISIS 3302 – Modelado, Simulación y Optimización.

Proyecto 1: Optimización en logística para la empresa LogistiCo

Entrega 1-Implementación

1. Carga de datos:

El proceso de carga de datos se basa en varios pasos, inicialmente se descubren los archivos dentro de las carpetas “proyecto_caso_base” y “Project_b” aceptando variaciones por si los archivos se guardan de una manera diferente, es decir, si usa mayúsculas o minúsculas igualmente se detectan las dos carpetas. Dentro de cada una de ellas se buscan archivos con diferentes extensiones para evitar problemas de compilación (.csv, .xlsx, .ect.). e inmediatamente se lee cada archivo.

Luego de leer el archivo se hace una normalización inicial de los nombres de las columnas eliminando espacio en extremos, para evitar problemas a futuro.

Para cada tipo de archivo se aplica una limpieza y validación correspondiente transformando los datos si es necesario. Con respecto a clientes.csv se estandariza el id de la forma pedida en formato “C####” verificando que sean únicos, para la longitud y latitud se convierte a numérico los campos correspondientes, se extraen los números decimales con coma transformándolos a decimales con punto, para las cadenas de texto que tenían las medidas solo se tomaron los números necesarios y finalmente se revisa el rango pedido y se hacen las transformaciones necesarias para que estas estén dentro de [-90,90] y finalmente se revisa si hay una ventana para los datos de los casos 2 y 3 normalizando el formato a “HH:MM-HH:MM” y los valores vacíos o inválidos se remplazaban por “00:00:-23:59” es decir que la ventana era todo el día. Para los archivos vehicle.csv, depots.csv y parameters.csv también se hace la estandarización correspondiente de los ids verificando duplicados y se convierten todas las columnas numéricas a formato numérico. Y finalmente se eliminan las filas totalmente vacías o duplicados exactos.

Finalmente se guardan los datos limpios en nuevas carpetas y se genera un reporte con el nombre “integrity_report.json” donde podemos ver que ninguno de los archivos tiene issues por lo que están listos para ser utilizados en los diferentes casos.

2. Caso1:

a. Implementación:

Inicialmente se intentó seguir el modelado de la entrega pasada, sin embargo, se hicieron ciertos ajustes para que los solvers lo pudieran resolver en poco tiempo y de manera óptima, para esto se hace una pre-heurística para estimar los costos cliente-vehículo, luego se hace un modelo pyomo pequeño para asignar los clientes a un vehículo, se reconstruyen y se optimizan las rutas reales y finalmente se calcula el costo final real y se generan los resultados.

El código inicia cargando los datos previamente limpiados en la carga de datos luego se hace una pequeña estandarización para verificar que las coordenadas sean de tipo float, los ids tengan el formato correspondiente y se calculan las distancias con la formula Haversine, también se construye una matriz de distancias incluyendo el deposito como nodo 0.

Para lograr simplificar el modelo en pyomo de tal manera que este funcionara correctamente se hizo una pre-heurística para estimar los costos que representa asignar un cliente a un vehículo en particular, por lo que para cada vehículo se toma un cliente como semilla y mientras haya capacidad se va llenando la ruta, luego se calcula y costo real de la ruta y finalmente se reparte este costo entre los clientes que aparecen en la misma creando una matriz con costos y distancias aproximadas.

EL modelo pyomo implementado conserva lo esencial y se optimiza $Z[c,v]$ y $Y[v]$, es decir, si el vehículo v atiende al cliente c y si el vehículo v se usa respectivamente. La función objetivo quedó muy parecida intentando minimizar los costos con las restricciones de que solo un vehículo puede atender a un cliente en específico, la capacidad real aproximada de cada vehículo y la autonomía aproximada de cada vehículo.

Finalmente, con la asignación de pyomo para cada vehículo se toma su conjunto de clientes asignado y se ordenan usando Nearest Neighbor para poder tener una ruta asica por vehículo y luego se pasa a un proceso de corregir la secuencia para reducir la distancia real aplicando 2-opt que se especializa en mejorar los arcos con el fin de acortar distancias, luego se intenta mover el orden de los clientes de la misma ruta para buscar mejoras y mueve un cliente entre vehículos para verificar que los costos no disminuyan más de lo que ya están, o si es así se cambia la ruta y por último se guardan todos los resultados.

b. Resultados:

En la figura 1 podemos observar el resumen de la consola en donde se puede observar que toma en cuenta todos los clientes y vehículos en los datos y para esta cantidad de datos genera 3 rutas con 3 vehículos distintos en donde el costo final minimizado es igual a 821,097 COP y la distancia total recorrida fue de 165.98 km.

Los vehículos son V001, V002 Y V008 en donde en ningún momento superan su capacidad de carga o autonomía, por lo que se puede evidenciar que todas las rutas propuestas son posibles y factibles.

```
(pyomo.env) C:\Users\Admin\Desktop\proyecto_moos_etapa2>python casol.py
INFO: Almacenamiento cargado desde pickle.
INFO: Clientes: 24, Vehículos: 8
INFO: Capacidades: [130.0, 140.0, 120.0, 100.0, 70.0, 55.0, 110.0, 114.0]
INFO: Precomputando costos estimados C_est[[c,v]] usando heurística semilla por cliente...
INFO: Precomputación completada.
INFO: Resolviendo modelo Pyomo (asignación) con GLPK...
INFO: Resultado solver: ok, terminación: optimal
INFO: Optimizando rutas con 2-OPT + recolocaciones intra/inter...
INFO: Optimización local finalizada.
INFO: Guardados CSV verificaciones\casol\verificacion_casol.csv y JSON verificaciones\casol\verificacion_casol_resumen.json
INFO: Resumen final: vehículos usados=3, distancia total=165.98 km, costo total=821097 COP
Resumen: {'routes': [{'VehicleId': 'V001', 'InitialLoad': 130, 'RouteSequence': 'CD01 - C020 - C012 - C021 - C019 - C001 - C024 - C018 - C003 - C023 - C014', 'ClientsServed': 10, 'DemandsSatisfied': '15-12-14-11-13-11-12-12-15-15', 'TotalDistance': 59.421, 'TotalTime': 89.13, 'FuelCost': 32286, 'OperationCost': 298261}, {'VehicleId': 'V002', 'InitialLoad': 140, 'RouteSequence': 'CD01 - C008 - C005 - C017 - C016 - C010 - C015 - C004 - C022 - CD01', 'ClientsServed': 8, 'DemandsSatisfied': '20-20-25-10-15-17-15-18', 'TotalDistance': 49.367, 'TotalTime': 74.05, 'FuelCost': 26823, 'OperationCost': 249606}, {'VehicleId': 'V008', 'InitialLoad': 107, 'RouteSequence': 'CD01 - C007 - C002 - C013 - C006 - C009 - C011 - CD01', 'ClientsServed': 6, 'DemandsSatisfied': '17-15-21-17-20-17', 'TotalDistance': 57.188, 'TotalTime': 85.78, 'FuelCost': 31072, 'OperationCost': 281230}], 'total_cost': 821097, 'total_distance': 165.976, 'vehicle_count': 3, 'total_fuel_cost': 90181, 'total_fixed_cost': 150000, 'total_variable_cost': 580916, 'solver_used': 'glpk', 'warnings': []}]
```

Ilustración 1

En la figura 2 podemos ver más a detalle las rutas por cada vehículo y cliente, en donde la línea de color azul representa el vehículo 1 el cual visita los clientes: c020 - c012 - c021 - c019 - c001 - c024 - c018 - c003 - c023 - c014 partiendo y volviendo desde CD01, la línea naranja representa el vehículo 2 en donde desde CD01 hasta volver a este punto visita los clientes: c008 - c005 - c017 - c016 - c010 - c015 - c004 - c022 y finalmente la línea verde representa el vehículo 8 que recorre la siguiente ruta: cd01 - c007 - c002 - c013 - c006 - c009 - c011 - cd01.

Analizando cada ruta, podemos evidenciar que las restricciones se cumplen a la perfección ya que no existe un cliente que sea visitado por dos vehículos, todos los vehículos parten y vuelven al mismo punto y como lo mencionamos anteriormente no se supera ni la capacidad ni autonomía de ningún vehículo.

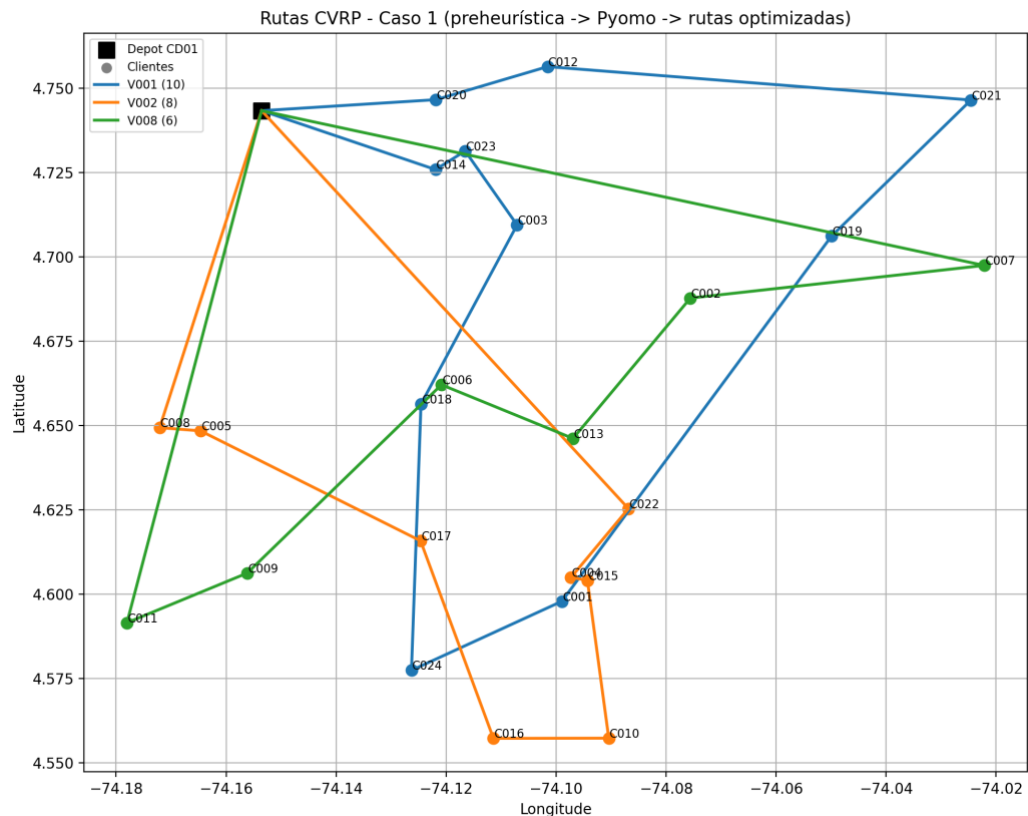


Ilustración 2

c. Conclusiones:

En conclusión, se implementó un modelo pyomo reducido en donde inicialmente se hace una heurística previa para poder manejar la complejidad de los costos aislada y que el modelo glpk de pyomo no fallara por una sobrecarga de memoria, y se hace una reconstrucción de los caminos después de ejecutar el modelo para poder calcular las distancias y costos reales de cada ruta.

Por lo visto en los resultados podemos observar que el código se desarrolla de una buena manera ya que no incumple ninguna restricción básica, sin embargo, pensamos que puede ser posible mejorar los resultados si se cuenta con un solver de pyomo más robusto que logre modelar la fórmula de costoso tan compleja que se había planteado en la entrega anterior.

3. Caso2:

A. Implementación:

Se define el modelo creando las variables binarias para los recorridos de los vehículos, además de los tomadores de decisiones para el cliente, vehículo y el viaje, uso del vehículo, tiempos de llegada y cargas. Para los parámetros: demanda, ventanas de tiempo, tiempos de servicio, horas de inicio, matrices de distancias, costos diferenciados por tipo y consumos.

En cuanto las restricciones que se detallan en el documento anterior también fueron agregadas al modelo, estas son: cada cliente una vez, flujo in/out ligado a z , salida y retorno al depósito si $y[v]=1$; capacidad y rango por vehículo, $z \leq y$, ventanas de tiempo duras, tiempos de depósito fijados a `start_time`;, precedencia temporal en arcos, evitar los ciclos.

Para la función objetivo se hace la suma de : vehículo, costos por KM, costo por hora (viaje + servicio), la energía del dron o combustible.

Apartir de los valores óptimos, se reconstruyen las secuencias por vehículo esto buscando los arcos desde el depósito, además calcula la distancia, tiempo, carga, costos por vehículo la llegada y las coordenadas.

B. Resultados:

En consola se imprimen los resultados como se muestra en la ilustración 3. La solución cumple las restricciones del escenario B: en primer lugar cada cada tour inicia y termina en el depósito y no presenta subciclos; segundo cada cliente es atendido exactamente una vez; tercero la suma de demandas por viaje respeta la capacidad declarada (camion: 167/200; dron: 18/25); cuarto la distancia total de cada tour está por debajo de la autonomía asignada al vehículo; quinto las horas de llegada respetan la propagación de tiempos (viaje + servicio) y las ventanas activando esperas cuando llega antes; Por ultimo el costo de cada ruta coincide con la función objetivo, separando combustible para camioneta y energía para dron.

```
Rutas encontradas (Caso 2):
- V001 (Truck): CD01-C008-C011-C003-C004-C002-C012-C001-C010-C005-C014-C013-CD01
ArrivalTimes: 08:45-08:51-10:15-13:00-13:33-13:40-13:50-14:30-14:56-15:02-15:10 | Distance km: 26.054 | Time min: 86.26
Cost: 198441.37 (Fuel 48777.58, Energy 0.0) | LoadCap: 200.0 | FuelCap: 92.345
- V003 (Drone): CD01-C006-C007-C009-CD01
ArrivalTimes: 09:00-09:11-14:18 | Distance km: 10.317 | Time min: 19.59
Cost: 39776.81 (Fuel 0.0, Energy 870.75) | LoadCap: 25.0 | FuelCap: 0.0
```

Ilustración 3

El mapa de la ilustración 4 muestra una asignación híbrida bien separada: el dron marcado en naranja atiende un cluster compacto al depósito, con un lazo corto alrededor de éste, típico de demandas livianas y ventanas tempranas que conviene despachar rápido y sin consumir mucha autonomía. La camioneta representada en azul recorre un perímetro más amplio, consolidando varios puntos dispersos en una sola ruta, justo donde su mayor capacidad y costo por km más estable rinden mejor. Ambas trayectorias parten y regresan al depósito y casi no se solapan, lo que sugiere una planificación que minimiza cruces.

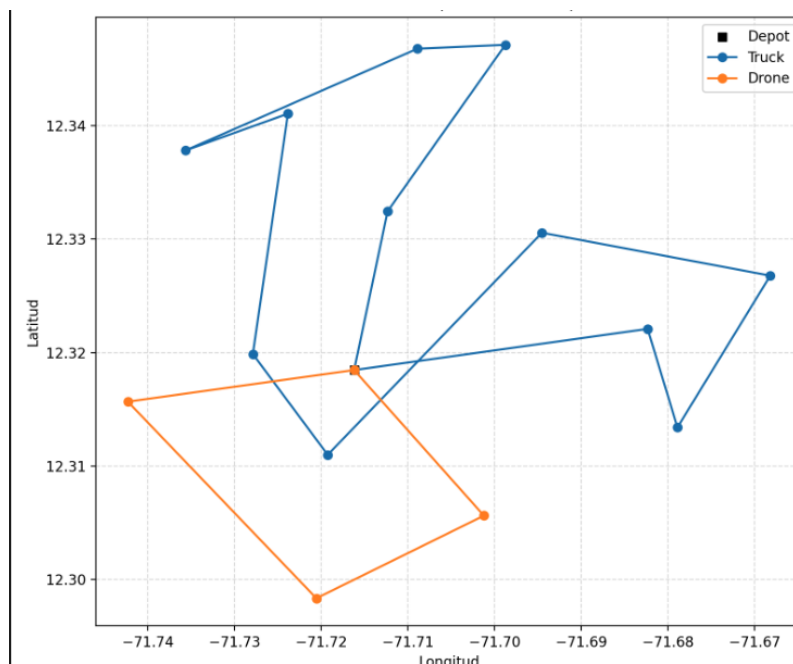


Ilustración 4

En el diagrama de Gantt (ilustración 5) las barras grises muestran las ventanas de tiempo de cada cliente y los puntos indican el momento real de llegada, azul representa a la camioneta y naranja al dron. En el gráfico todos los puntos caen dentro de su barra o ligeramente antes del inicio de la barra, lo que implica cumplimiento total de ventanas y, en esos casos adelantados, solo espera hasta que la ventana abra. Se ve además la lógica híbrida: el dron resuelve ventanas tempranas y cercanas y una cita más tarde aún dentro de su ventana, mientras la camioneta consolida la franja media-tardía, sin violar ventanas. En síntesis, el plan equilibra puntualidad y consolidación: usa el dron para absorber compromisos sensibles en tiempo y deja a la camioneta los recorridos más largos con varias paradas.

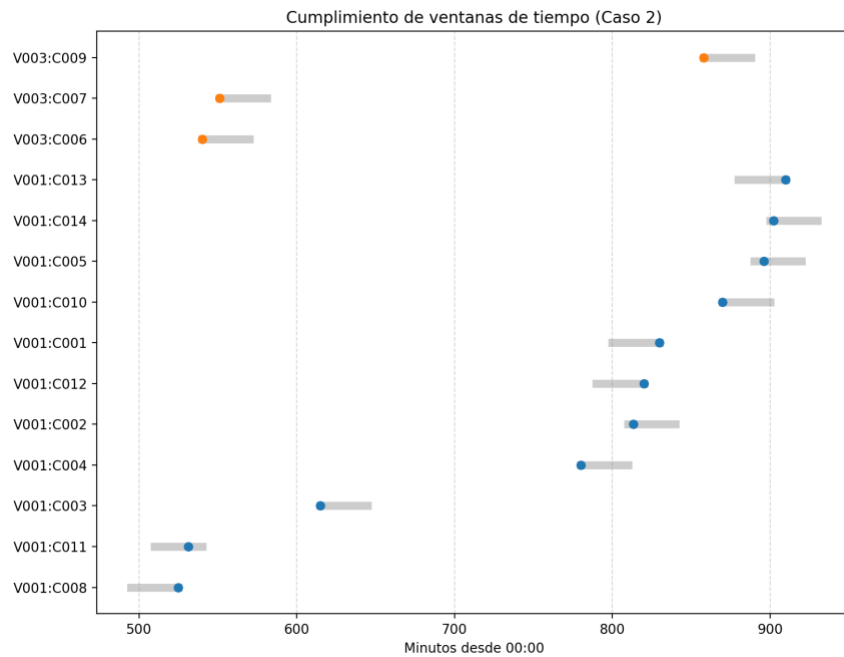


Ilustración 5

C. conclusiones:

En conjunto, los resultados confirman que la asignación híbrida está bien diseñada y cumple todas las restricciones: en el mapa, el dron concentra un clúster compacto cercano al depósito mientras la camioneta cubre un perímetro más amplio con mínimas intersecciones; en el gráfico de ventanas, todos los puntos caen dentro del tiempo esperado. El resumen por consola respalda esto con métricas clave: ambas rutas salen y regresan al depósito; las demandas atendidas coinciden con la carga inicial (V001: 167/200 kg, V003: 18/25 kg), las distancias son moderadas y compatibles con el rango (Truck: 26.054 km, Drone: 10.317 km), y los tiempos de desplazamiento son 86.26 y 19.59 min, respectivamente. Además, el desglose de costos es coherente con la FO: la camioneta refleja componente de combustible (COP 48,777.58) y el dron de energía (COP 870.75), con costos totales de 198,441.37 COP (Truck) y 39,776.81 COP (Drone). En síntesis, el plan usa al dron para absorber citas tempranas cerca del depósito y a la camioneta para consolidar visitas dispersas, manteniendo capacidad, rango y puntualidad.

4. Caso3:

A. Implementación:

Para encontrar una solución al caso 3, se realizó una implementación en Pyomo del caso 2, agregando las nuevas restricciones y utilizando el solver de HiGHs. Sin embargo, esta implementación no logra resolver el caso en un tiempo óptimo y de utilizar parámetros

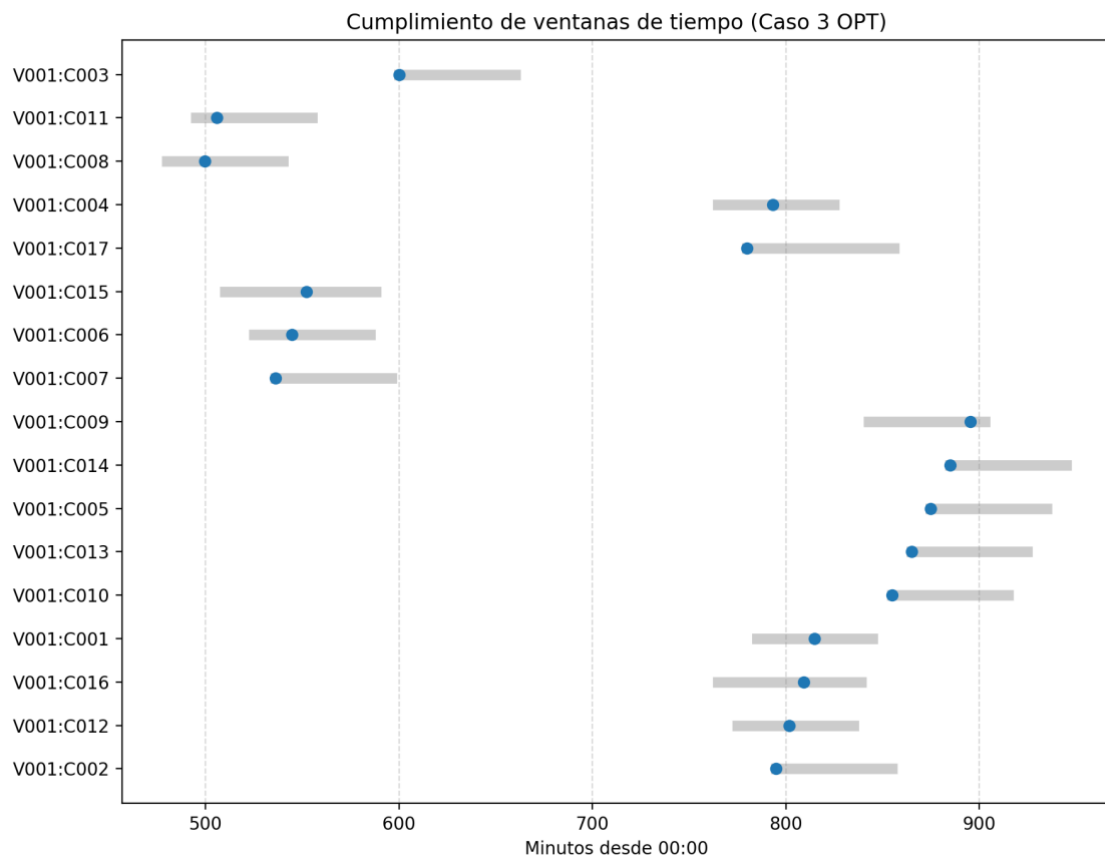
más bajos retorna que es infeasible. Es por esto, que se decidió implementar otra aproximación al caso 3 utilizando GAI (caso3_AI.py) la cuál utiliza una aproximación 2-OPT de TSP y además usa KNN. Esto únicamente para ser usado como referencia y no como nuestra solución propia ya que la implementada en PYOMO no arrojó resultado.

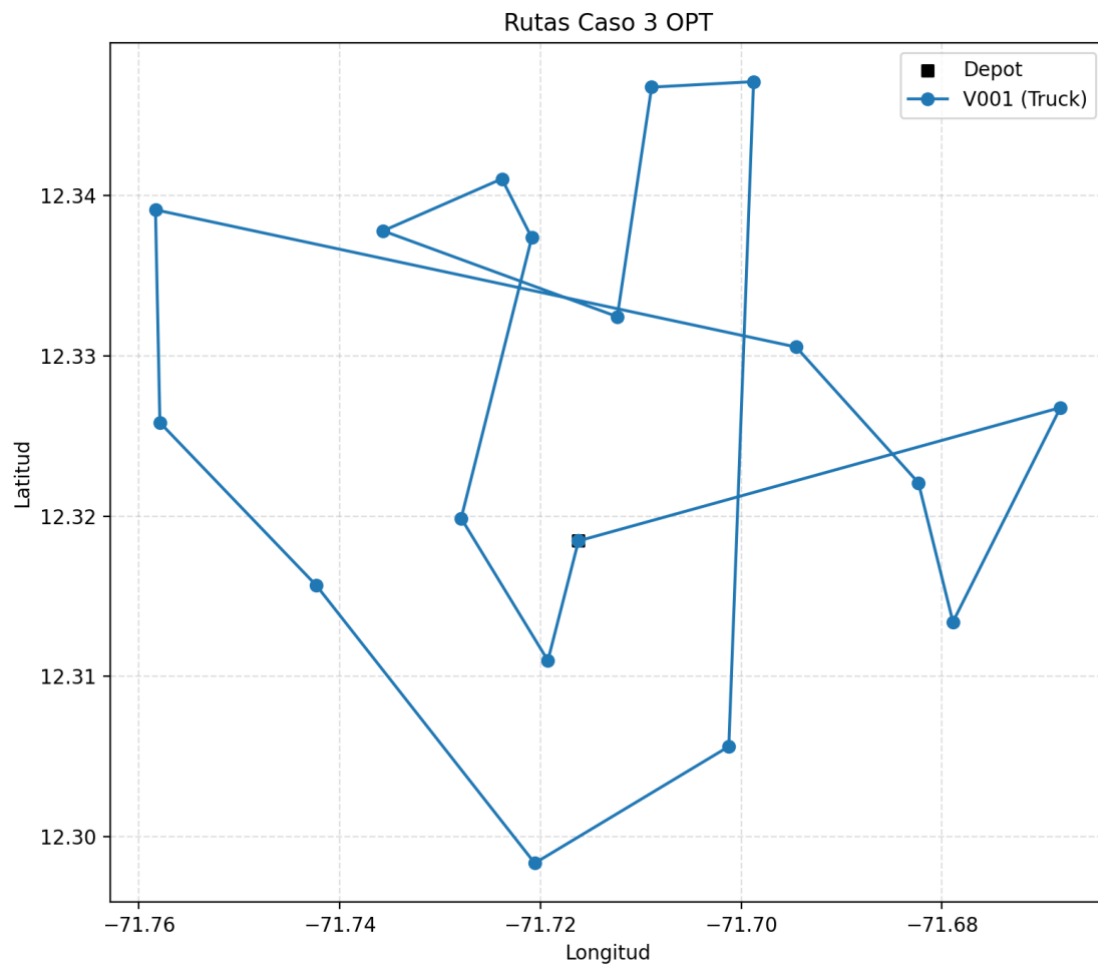
B. Resultados:

Como se puede observar, la implementación en Pyomo solamente se queda aquí cargando y no arroja un resultado debido a su alta complejidad.

```
/caso3
(base) david@Davids-MacBook-Air-5 Proyecto1-entrega2 equipooptimo % /opt/anaconda3/bin/python
-entrega2 equipooptimo/caso3.py
INFO: Cargando datos de Caso 3 Pyomo optimizado...
```

Por otro lado, podemos observar la solución arrojada por la implementación hecha con GAI en el archivo verificación_caso3_opt.csv. Además, de las siguientes gráficas:





Las cuales nos arrojan un resultado óptimo para una sola camioneta siguiendo la ruta enmarcada en el gráfico.

C. Conclusiones:

El acercamiento que intentamos hacer a este caso usando pyomo no fue posible optimizarlo. El problema necesita hacer de implementaciones de algoritmos aproximados (en este caso 2-OPT) para poder encontrar una solución la cual, no necesariamente, es la óptima general ya que su complejidad no nos permite encontrar una respuesta más allá de una aproximación.